Yarmouk University

Hijjawi Faculty for Engineering Technology

Department of Computer Engineering

Graduation Project Report

Smart Pothole Detection System

Full Updated Report (Implementation-Centered, AI-Integrated)

Includes AI model training, deployment, and evaluation details

Students Names:

حمزة عبدالحكيم محمود الصافي - 2022980008

عمر اوسامه علي عواوده - 2021980053

Supervisor Name: دكتور محمود مساعدة

Semester: 2025/2026

Date: March 2026

Students' Property Right Declaration and Anti-Plagiarism Statement

We hereby declare that this graduation project report at Yarmouk University is our own work, except for quotations and referenced material that are properly acknowledged. This work has not been accepted for any other degree and is not concurrently submitted elsewhere.

This updated version is implementation-focused and reflects the currently running codebase for the Smart Pothole Detection System, including the AI model lifecycle from dataset preparation and training to Android TensorFlow Lite deployment and evaluation outcomes.

We have read and understood the university rules on assessment offences and plagiarism.

Students: Hamza Al-Safi, Omar Awawdeh

Table of Contents

Students' Property Right Declaration and Anti-Plagiarism Statement	1
List of Tables	3
List of Figures	4
Abstract	5
Chapter 1: Introduction	7
Chapter 2: Background	9
Chapter 3: Design	10
Chapter 4: Implementation	14
Chapter 5: Results and Discussion	18
Chapter 6: Economical, Ethic, and Contemporary Issues	19
Chapter 7: Project Management	20
Chapter 8: Conclusion and Future Work	22
References	23
APPENDIX A: User Manual	24

List of Tables

Table 1. Target users and key needs	9
Table 2. Current approaches versus limitations	9
Table 3. Implemented architecture stack	11
Table 4. Design specifications summary	11
Table 5. Design alternatives and decisions	12
Table 6. Design considerations table	13
Table 7. Design trade-offs in implementation	14
Table 8. Main API endpoints and authorization	15
Table 9. Implemented database schema overview	15
Table 10. Preliminary cost estimation and justification	19
Table 11. Schedule and time management	20
Table 12. Risk register and mitigation plan	21
Table 13. AI dataset composition and split summary	18
Table 14. Final AI model validation metrics	19

List of Figures

Figure 1. High-level implemented architecture	8
Figure 2. Backend processing and database flow	10
Figure 3. Dashboard modules and operations	17
Figure 4. Local deployment topology (Docker Compose)	17
Figure 5. Implemented database ER view	16

Abstract

Potholes are a persistent infrastructure and road-safety problem that causes vehicle damage, increases accident risk, and raises municipal maintenance costs. Traditional reporting methods, including manual surveys and complaint-based workflows, are often delayed and inconsistent. This project addresses that operational gap by providing an end-to-end pothole reporting and management system with an AI detection engine, production-oriented backend, geospatial database, Android field client, and web dashboard.

The implemented platform receives camera frames on Android devices, performs on-device pothole detection using a YOLOv8n model exported to TensorFlow Lite float16, and converts high-confidence detections into geotagged reports. Backend services validate and deduplicate these reports using spatial logic, store records in PostgreSQL/PostGIS, and expose secured APIs for maintenance operations. The dashboard supports map-based visualization, filterable lists, detailed record editing, status workflow updates, bulk actions, vehicle management, user management, CSV export, and analytics summaries. The system is containerized through Docker Compose for consistent local deployment and testing.

Development followed an iterative approach over multiple project phases, with continuous integration of AI training artifacts, Android inference logic, backend services, data model refinements, and dashboard usability improvements. Final deliverables include a trained YOLOv8n model (mAP@50 = 80.52%, mAP@50-95 = 50.29%), Android on-device inference pipeline, implemented API server, database schema/migration, operational dashboard, runtime orchestration files, and this updated technical report. We acknowledge our supervisor and Yarmouk University for technical guidance and academic support throughout the project lifecycle.

Keywords: YOLOv8n, TensorFlow Lite, On-device inference, Pothole management, Geospatial deduplication, ASP.NET Core API

Chapter 1: Introduction

1.1 Problem Statement and Purpose

Urban roads degrade continuously, and potholes may spread faster than maintenance teams can inspect and repair them. Manual inspection workflows are labor-intensive and do not provide continuous, city-scale visibility. This project aims to provide a practical AI-enabled software system that transforms pothole detections into structured operational data, then into actionable maintenance decisions.

1.2 Background

Conventional road-defect management usually relies on periodic field visits and citizen reporting. While both are valuable, they are not sufficient for proactive maintenance planning. A data-driven, location-aware workflow is needed to prioritize repairs and monitor status changes transparently.

1.3 Aims and Objectives

- Build a secure backend API for pothole ingestion and lifecycle management.
- Persist pothole records and geospatial attributes in PostgreSQL/PostGIS.
- Train and evaluate a mobile-ready pothole detection model using YOLOv8n.
- Deploy TensorFlow Lite inference on Android with real-time camera processing.
- Implement operational dashboard workflows for map/list/detail management.
- Support role-based access for admin, operator, and viewer users.
- Provide export and analytics features for municipal reporting.

1.4 Existing Ways the Problem Is Solved

Existing approaches include manual road surveys, public complaint channels, isolated digital tracking tools, and vision-only research prototypes disconnected from municipal operations workflows. These approaches improve reporting coverage partially, but often miss robust deployment constraints, deduplication, status workflows, and integrated geospatial analytics.

1.5 Main Solution Idea

The main idea is an implementation-centered AI operations platform: detect potholes on-device through a TensorFlow Lite model, convert detections into geo-tagged events, validate and deduplicate them server-side, store them in a geospatial database, then expose map/table/detail workflows for maintenance teams through a secure dashboard.

1.6 Key Technical Details and Evaluation Approach

- Backend stack: ASP.NET Core 8, Entity Framework Core, JWT, SignalR.
- Database: PostgreSQL 16 + PostGIS with geography point and indexed access paths.
- Dashboard: React + Vite + Tailwind + React Leaflet + Axios.
- AI stack: Ultralytics YOLOv8n trained in Colab and exported as float16 TFLite model.
- Android inference: model input [1,640,640,3], output [1,5,8400], confidence threshold 0.5.

- Deduplication: 15-meter radius to avoid duplicate pothole inflation.
- Evaluation basis: implemented feature completion, workflow correctness, and runtime behavior.

1.7 Contributions

- Implemented full REST API surface for auth, potholes, users, vehicles, and stats.
- Implemented geospatial deduplication and confirmation-count behavior in service layer.
- Prepared and trained AI dataset with 2,642 images and 9,077 annotations for single-class pothole detection.
- Integrated on-device TensorFlow Lite inference with NNAPI/GPU/CPU delegate fallback on Android.
- Implemented production-like dashboard workflows (list, map, detail, bulk, export, analytics).
- Implemented Docker-based local orchestration with health-aware service dependencies.
- Updated documentation and report content to reflect actual running implementation.

1.8 High-Level Figure

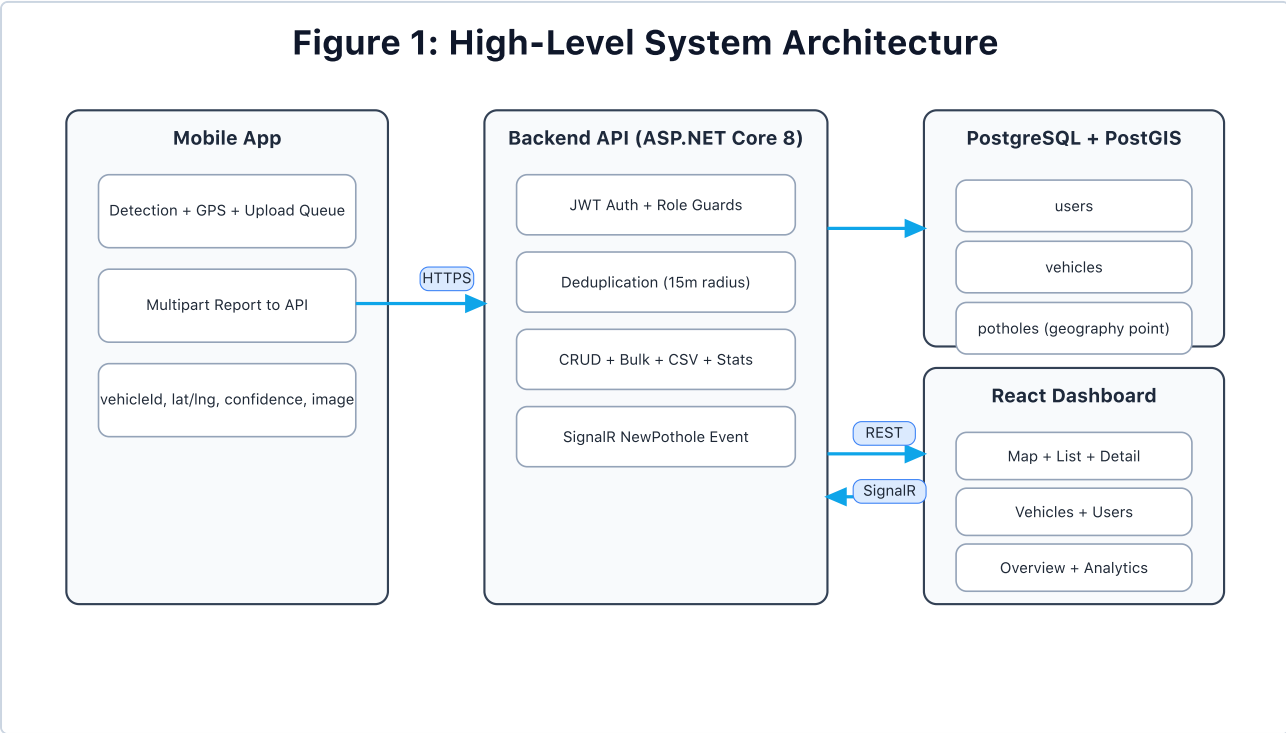

Figure 1. High-level implemented architecture

1.9 Summary of Report Structure

Chapter 2 discusses background and current approaches. Chapter 3 explains design choices and constraints. Chapter 4 describes implementation details across AI, Android, backend, and infrastructure. Chapter 5 discusses measured results and current strengths/limitations. Chapters 6 and 7 cover ethics, economics, and management. Chapter 8 concludes and outlines future work. Appendix A provides user manual guidance.

Chapter 2: Background

2.1 Background of the Problem (Literature-Oriented, Solution-Agnostic)

Road-defect monitoring has historically combined engineering inspections, complaints, and periodic surveys. As transportation loads increase, these methods struggle with scale, temporal coverage, and objective prioritization. Literature on smart-city maintenance consistently highlights the value of digitized incident records and geospatial workflows [1], [2].

2.2 Problem Context and Importance

- Safety impact: potholes increase accident risk and emergency response constraints.
- Economic impact: delayed repairs increase future resurfacing and public compensation costs.
- Operations impact: maintenance teams need clear, current, and verifiable defect data.

2.3 Target Market and Their Needs

Target Segment	Needs	Expected Value
Municipal maintenance units	Verified map/list of active potholes	Faster prioritization and dispatch
Operations supervisors	Status tracking and trend analytics	Data-driven planning and accountability
Fleet operators and municipal inspectors	Low-latency detection and simple reporting pipeline	Consistent road-condition contribution

Table 1. Target users and key needs

2.4 Potential Ethical and Environmental Issues

- Privacy risk if images incidentally capture sensitive objects or identifiable data.
- Security risk if location records are leaked or access controls are weak.
- Environmental impact of road deterioration and repeated resurfacing cycles.
- Opportunity for sustainability through better maintenance timing and route optimization.

2.5 Current/Previous Approaches

Approach	Typical Strength	Typical Limitation
Manual surveys	Direct field verification	High cost, low frequency, inconsistent coverage
Complaint-only channels	Simple participation	Subjective and uneven reporting density
Isolated digital logs	Basic record keeping	No unified geospatial dedup/workflow management

Approach	Typical Strength	Typical Limitation
Research-only CV demos	Strong algorithm focus	Weak operational integration and lifecycle tracking

Table 2. Current approaches versus limitations

Chapter 3: Design

3.1 Design Overview

3.1.1 System Design to Achieve Solution

The implemented design follows a service-oriented architecture where the backend API acts as the control point for validation, authorization, deduplication, persistence, and analytics exposure. The dashboard is designed for operational decisions, not only visualization. Upstream, an Android AI inference layer performs local pothole detection and emits confidence-scored candidate reports.

3.1.2 How the Design Addresses the Problem Statement

1. Transforms reports into normalized geospatial records.
2. Detects potholes directly from camera frames using optimized mobile AI inference.
3. Prevents duplicate inflation using server-side spatial distance checks.
4. Provides lifecycle status model for verification and repair tracking.
5. Supports role-separated actions to reduce operational misuse risk.
6. Adds export and analytics for planning and external reporting.

3.1.3 Detailed Figure and User Scenarios

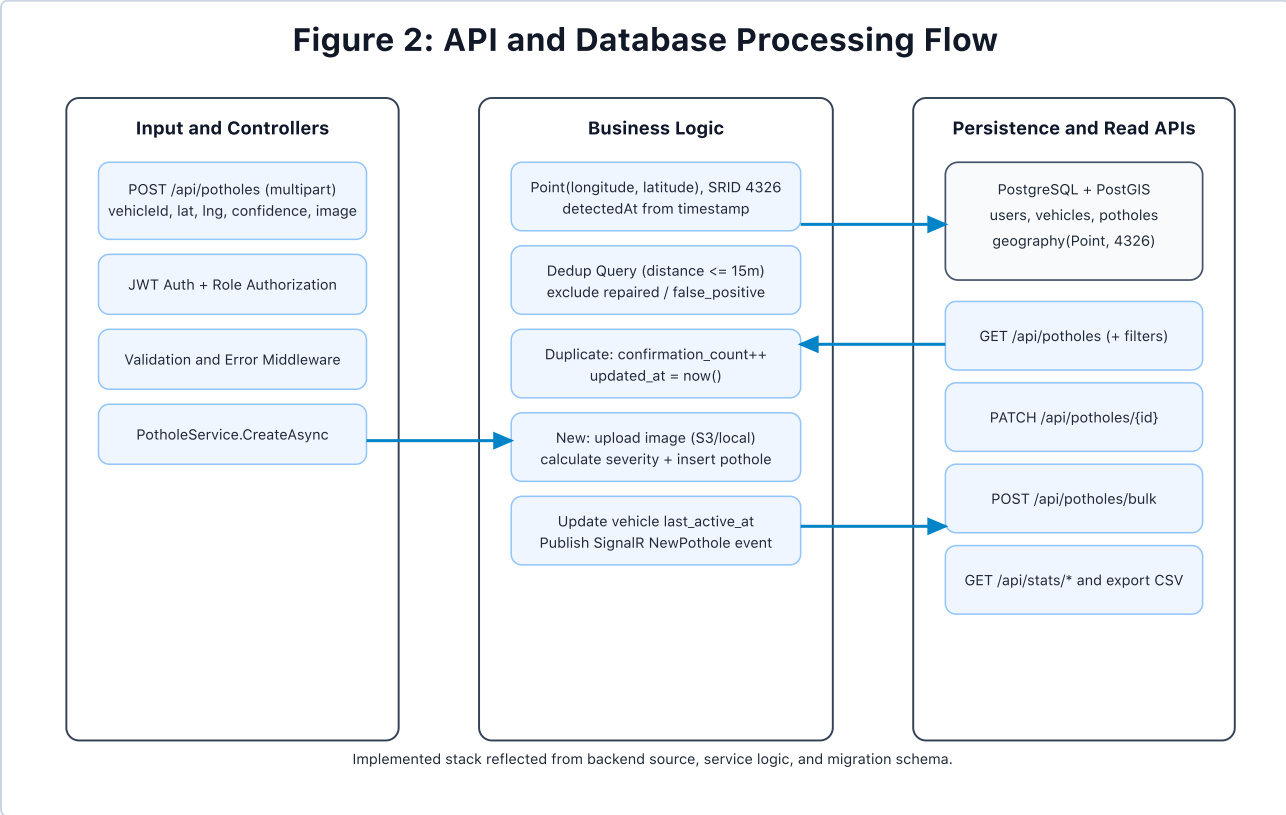

Figure 2. Backend processing and database flow

Scenario A (Operator): login, filter unverified points, open details, verify or mark false positive, then continue with next priority cluster.

Scenario B (Admin): review trends, bulk mark repaired after maintenance feedback, export CSV for daily reporting.

3.2 Design Details

3.2.1 Design Specifications

Dimension	Specification	Notes
Backend runtime	ASP.NET Core 8 API, containerized	Host port defaults to 3000
Dashboard runtime	React + Vite, containerized	Host port defaults to 4321
Database	PostgreSQL 16 + PostGIS 3.4	Host port defaults to 5432
Security	JWT auth + role guards + CORS policy	Admin/operator/viewer model
Geospatial handling	geography(Point,4326), 15m dedup radius	Dedup in service layer
Storage	S3 optional, local fallback	Environment-dependent
AI model	YOLOv8n - best_float16.tflite	~6 MB, on-device inference
AI training data	2,642 images, 9,077 labels	Train/valid/test split documented

Table 3. Implemented architecture stack

Design Aspect	Current Application	Constraint/Quality Goal
Performance	Indexed filters + bounded pagination	Responsive list/map interactions
Reliability	Container health checks and startup dependencies	Predictable local boot sequence
Security	JWT + role gates + centralized middleware	Least-privilege operations
Maintainability	Service/controller separation	Clear extension points
Usability	Map/list/detail workflow and bulk actions	Low friction for operators
AI reliability	Confidence + NMS + dedup across frames/locations	Reduce false alarms and duplicate reporting

Table 4. Design specifications summary

3.2.2 Design Process

The design process iterated through implementation checkpoints. First, schema and auth were stabilized. Next, pothole lifecycle APIs were implemented with dedup and severity mapping. Finally, dashboard pages were built around real API behavior, including map fit, filtering, bulk operations, and analytics. This sequence reduced rework by validating data contracts early.

3.2.3 Legal Aspects

Legal considerations include lawful handling of geolocation-linked records, controlled account access, and appropriate data retention policies. System design supports policy enforcement through role-based access, API authentication, and centralized backend validation.

3.2.4 Design Constraints

- Time and team constraints required prioritizing operational workflows over advanced optimizations.
- Infrastructure constraints favored a reproducible local stack before production-scale rollout.
- Privacy constraints required strict access control and minimal sensitive data exposure.

3.2.5 Design Standards

- Security best-practice guidance from OWASP and framework security recommendations [9], [10].
- Professional ethics context from IEEE and ACM codes [11], [12].
- Standards body references for regional/international compliance orientation (ISO/IEC/JSMO/NIST).

3.2.6 Design Alternatives

Alternative	Considered Benefit	Reason Not Chosen
No geospatial DB	Simpler schema	Cannot support distance-based dedup robustly
Single-page minimal UI	Lower development effort	Insufficient for operations workflow complexity
Non-containerized setup	Fewer setup files	Lower reproducibility across environments
Larger detector model (YOLOv8s)	Potentially higher recall	Higher latency and resource usage on mobile

Table 5. Design alternatives and decisions

3.2.7 Safety Consideration

- Operational safety: status workflows reduce false dispatch and improve maintenance prioritization.
- Data safety: authenticated access and role checks reduce accidental destructive operations.
- AI safety: confidence threshold and non-max suppression prevent unstable over-reporting.
- Team safety: local containerized development reduces risky ad-hoc environment configuration.

3.2.8 Design Considerations Table

Design Consideration	Project Application	Relevant Location
Performance	Pagination, indexed querying, compact API payloads	3.2.1, 4.1, 4.2
Serviceability	Containerized services and modular code structure	3.2.1, 4.1, 4.4
Economic	Open-source stack, low-cost deployment path	6.1, 7.2
Environmental	Maintenance optimization reduces repeated repairs	6.4
Environmental Sustainability	Data-driven repair planning reduces wasteful operations	6.4, 8.2
Manufacturability	Software-only deployability with Docker images	4.4
Ethical	Role-based data access and privacy-aware operations	6.2, 6.3
Health and Safety	Road hazard visibility supports safer travel decisions	2.2, 6.5
Social	Improved road maintenance transparency and quality	2.2, 6.5
Political	Supports evidence-based municipal planning and accountability	6.5

Table 6. Design considerations table

Chapter 4: Implementation

4.1 Methods and Tools (Hardware, Software)

Software stack: Ultralytics YOLOv8n (training), TensorFlow Lite (inference), Android Kotlin, ASP.NET Core 8, Entity Framework Core, Npgsql + NetTopologySuite, JWT bearer auth, SignalR, React 19, TypeScript, Vite, Tailwind, React Leaflet, Axios, Docker Compose.

Runtime tools: PostgreSQL/PostGIS container, health checks, mounted upload volume, environment-driven ports.

4.2 Infrastructure Requirements

- Container runtime (Docker Engine + Compose plugin).
- PostGIS-compatible PostgreSQL instance.
- Backend container exposing API + hub + static upload path.
- Frontend container proxying API requests to backend service.
- Optional object storage integration for production (S3).

4.3 Trade-offs in Design and Implementation

Trade-off	Choice Made	Impact
Dedup complexity vs data quality	15m geospatial dedup	Reduces duplicate noise with moderate compute cost
Simple auth vs role granularity	JWT + role guards	More secure operations, slightly more logic
Local-only storage vs cloud option	Storage abstraction	Flexible deployment with manageable complexity
Minimal UI vs operations UI	Multi-page workflow dashboard	Higher dev effort, better real operations usability
Model size vs detection capacity	YOLOv8n float16 (~6MB)	Good speed on mobile with acceptable recall trade-off

Table 7. Design trade-offs in implementation

4.4 Dependencies and Assumptions

- Valid report payload includes vehicle ID, coordinates, confidence, and timestamp.
- Operational users are assigned correct roles (admin/operator/viewer).
- Database healthcheck passes before backend startup in compose deployment.
- Frontend has network access to backend API/hub through configured proxy.

4.5 Implemented API Surface

Controller Area	Implemented Endpoints	Authorization
Auth	login, register, refresh, logout, me, profile	Mixed public/auth/admin
Potholes	create, list, get, update, delete, bulk, export	Authenticated + role-restricted actions
Vehicles	create, list, get, update, delete, vehicle-potholes	Authenticated + admin write actions
Users	list, create, get, update, delete	Admin-only
Stats	overview, daily, by-status, by-vehicle, heatmap	Authenticated

Table 8. Main API endpoints and authorization

4.6 Implemented Database Schema

> Implementation note: `vehicles` is intentionally not linked to `users` in the current schema.

Table	Key Fields	Purpose
users	id, email unique, password_hash, role, timestamps	Identity and authorization
vehicles	id, name, serial_number unique, active/last_active fields	Report source management
potholes	vehicle_id FK, lat/lng, geography point, status/severity, confirmation_count	Core pothole lifecycle and geospatial operations

Table 9. Implemented database schema overview

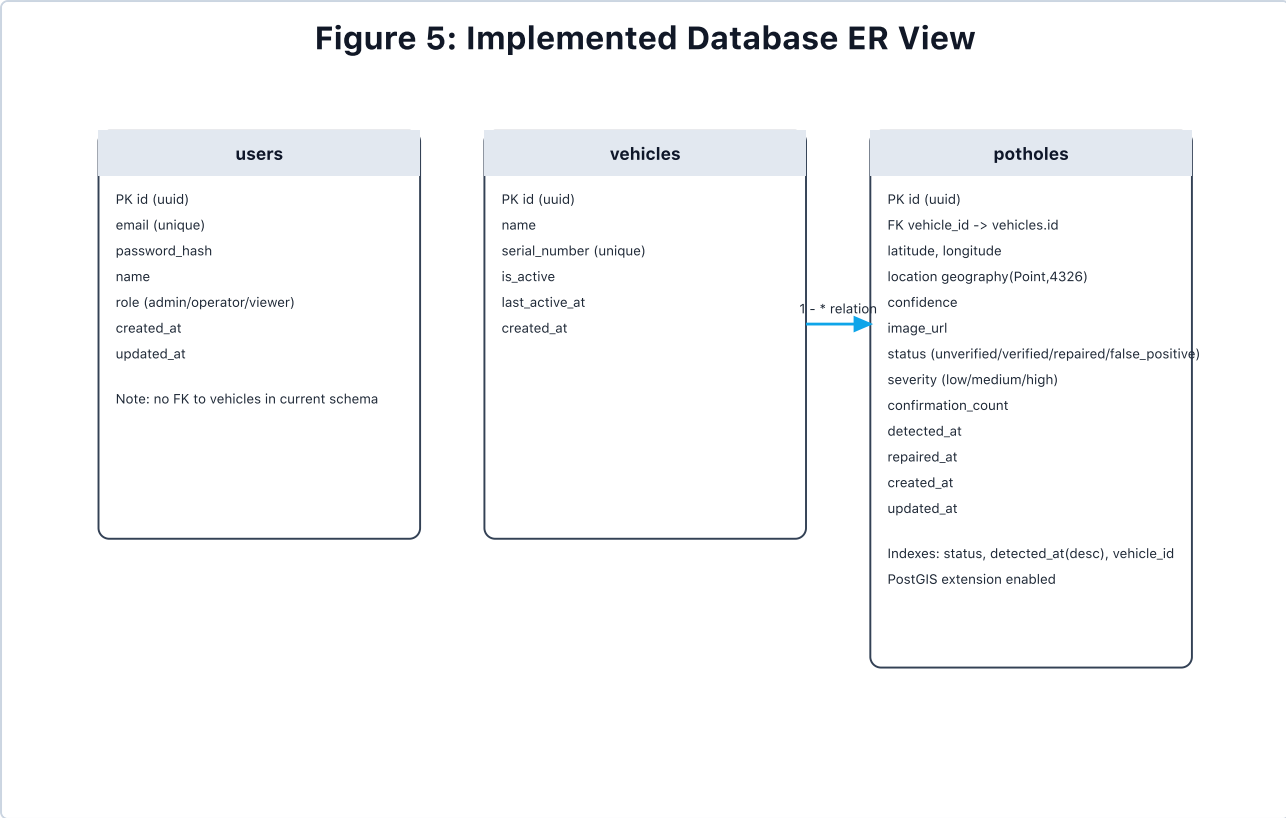

Figure 5. Implemented database ER view

4.7 Dashboard Functional Implementation

- Login page and token lifecycle handling with refresh-on-401 behavior.
- Protected routes and role-aware operations in management pages.
- Pothole list with filters, pagination, bulk actions, and CSV export.
- Map page loading all points, fitting bounds, and optional heatmap overlay.
- Pothole detail page with status/severity update and delete support.
- Vehicle and user management pages for administrative operations.
- SignalR listener for NewPothole notifications in dashboard UI.

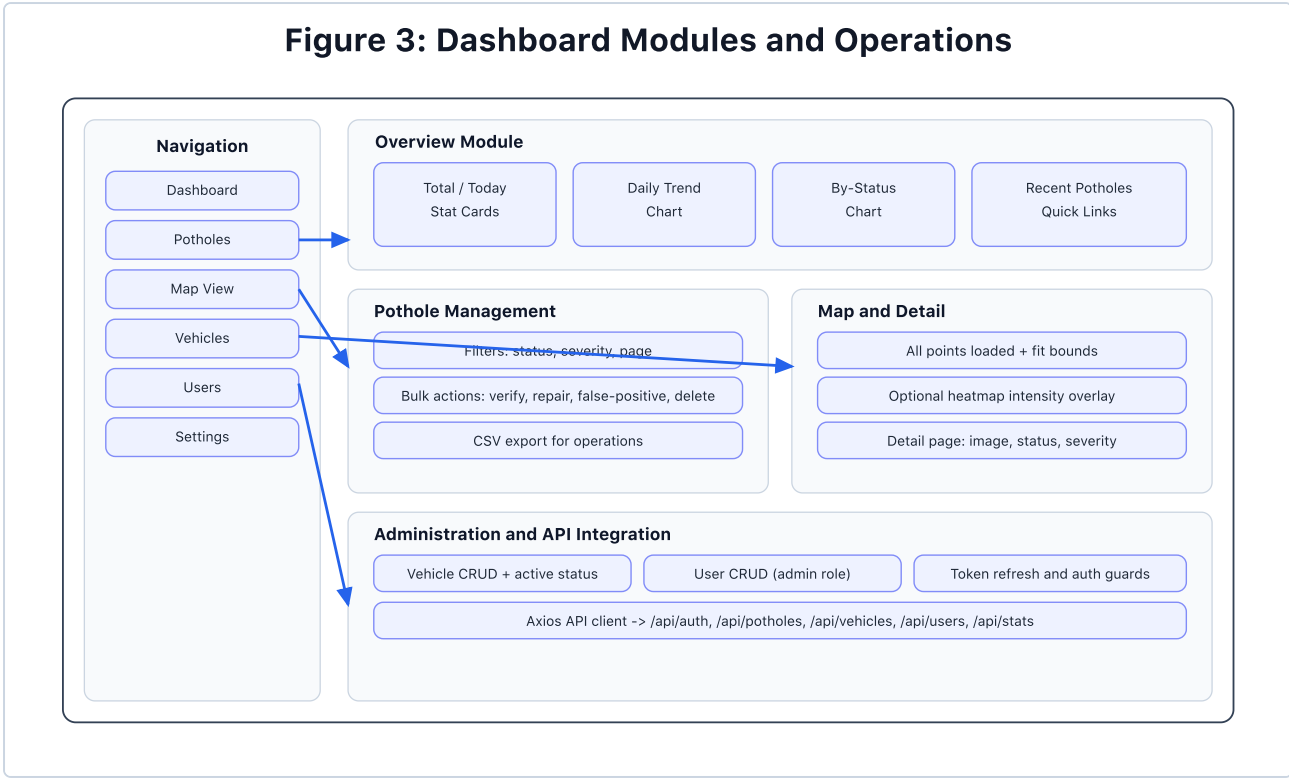

Figure 3. Dashboard modules and operations

4.8 AI Dataset, Training, and Model Export

AI development was implemented in the `ai-model/` workspace using a reproducible Colab workflow and documented scripts/utilities. The final dataset contains 2,642 labeled images and 9,077 pothole instances from Roboflow and Kaggle sources. Training used YOLOv8n with 100 epochs, AdamW optimizer, and augmentation settings tuned for road-scene variation.

Split	Images	Annotations	Ratio
Train	2,129	7,565	80.6%
Validation	382	1,116	14.5%
Test	131	396	5.0%

Table 13. AI dataset composition and split summary

All quantitative metrics in this report are anchored to the saved artifacts under `ai-model/trained_model/` (especially `results.csv` and `TRAINING_METRICS.md`). The trained model was exported to `best_float16.tflite` and integrated into Android assets. Model I/O specification is fixed to input shape `[1, 640, 640, 3]` and output shape `[1, 5, 8400]`. This format supports fast on-device inference and avoids network dependency for core detection functionality.

4.9 Android AI Inference and Reporting Pipeline

The Android inference path is implemented in `PotholeDetector.kt` and `ProcessFrameUseCase.kt`. Frames are resized/normalized, inference is executed through TensorFlow Lite with delegate fallback (NNAPI, GPU, CPU), candidate boxes are filtered by

confidence, NMS is applied, and resulting detections are passed to reporting logic. Reporting then performs GPS tagging, deduplication checks, and queued upload to the backend.

4.10 Deployment Topology

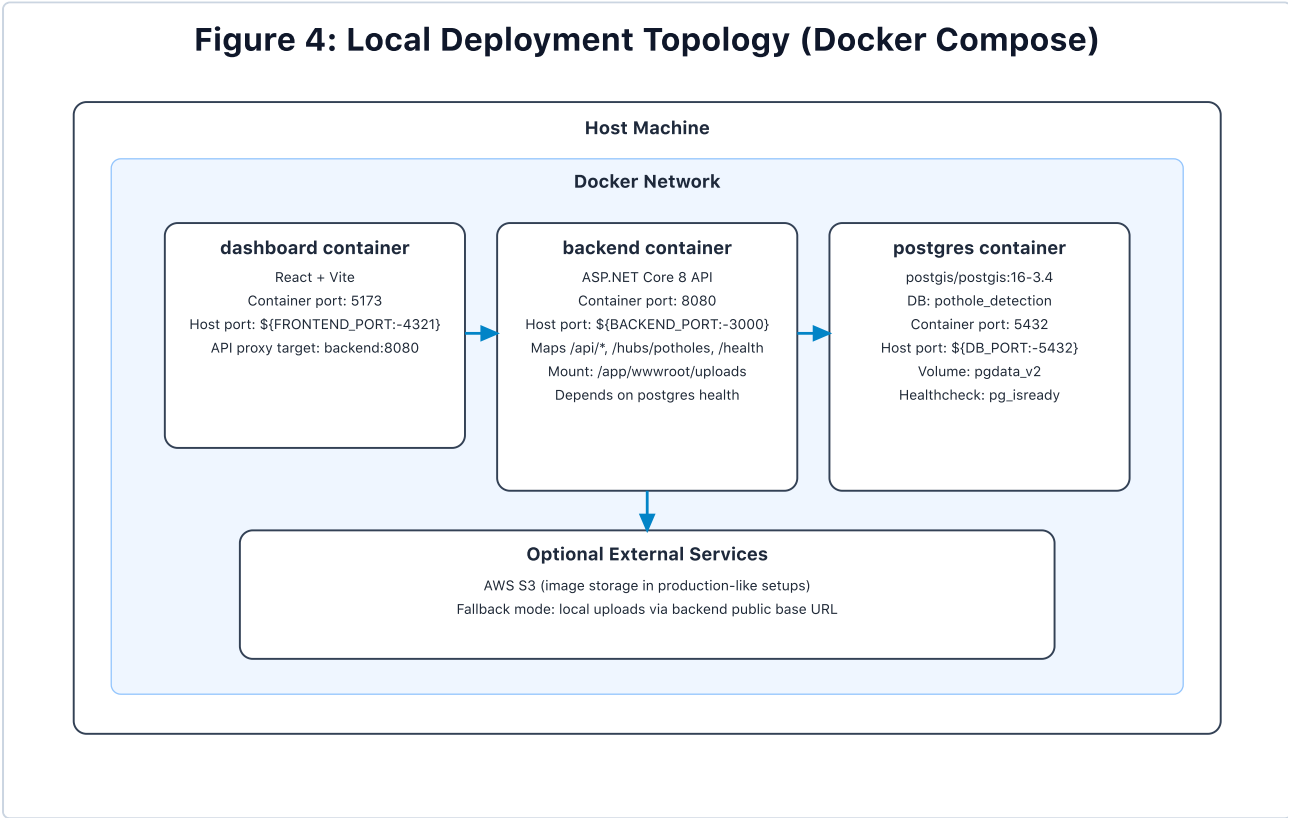

Figure 4. Local deployment topology (Docker Compose)

Chapter 5: Results and Discussion

5.1 Results of Implemented Work and Link to Design

The implementation validates major design objectives from Chapter 3: secure ingest, geospatially aware deduplication, operational dashboard workflows, and reproducible deployment. The backend services and schema definitions now directly support lifecycle actions (unverified, verified, repaired, false positive) and analytics endpoints needed for maintenance reporting.

- Dedup logic is operational in service layer and tied to geography distance checks.
- AI model training achieved target metrics and was successfully exported to mobile TFLite format.
- Android on-device inference operates without mandatory cloud inference dependency.
- Status updates and bulk actions provide practical field-to-office workflow continuity.
- Heatmap and stats endpoints are integrated into dashboard visualizations.
- Role-based controls restrict sensitive operations to intended user classes.

5.2 AI Quantitative Evaluation

Metric	Achieved Value	Project Target	Interpretation
mAP@50	80.52%	>80%	Strong detection quality at IoU 0.5
mAP@50-95	50.29%	>50%	Robust precision across strict IoU thresholds
Precision	81.16%	>80%	Low false-positive behavior for operations
Recall	71.77%	>70%	Detects majority of potholes with conservative threshold
Model Size	~6 MB	<10 MB	Fits mobile constraints well
Estimated Mobile FPS (not benchmarked)	25-30 FPS	>20 FPS (goal)	Estimate from model/deployment expectations, not direct benchmark in this report

Table 14. Final AI model validation metrics

These results show that the selected YOLOv8n configuration meets performance targets while preserving mobile deployability. The achieved precision is particularly useful in operational contexts where false alerts can waste maintenance effort.

5.3 Strengths and Weaknesses

Strengths:

- Documented and reproducible AI training process with dataset and metrics artifacts.
- Efficient on-device inference architecture with delegate fallback.
- Clear modular architecture between controllers, services, and data model.
- Practical geospatial model with index support and dedup behavior.

- Operationally useful UI (map + list + detail + bulk + export).
- Containerized startup path suitable for development and staging.

Weaknesses / Current Limitations:

- Recall can be improved further for safety-critical high-sensitivity deployments.
- Current AI model supports single-class detection (pothole only).
- No full audit trail for every status transition and actor action yet.
- No advanced route optimization for maintenance dispatch in current release.
- Cloud production hardening and observability still require additional work.

Chapter 6: Economical, Ethic, and Contemporary Issues

6.1 Preliminary Cost Estimation and Justification

Item	Justification	One-Time Cost (JOD)	Recurring Cost (JOD/Month)
Server/hosting setup	API + DB + dashboard operation	0	20-35
Domain and TLS operations	Public secure access in production	10-15 yearly equivalent	1-2
Storage (optional S3)	Scale image retention	0	3-12 (usage dependent)
Development toolchain	Open-source stack	0	0

Table 10. Preliminary cost estimation and justification

6.2 Relevant Codes of Ethics and Moral Frameworks

- IEEE Code of Ethics: safety, transparency, and responsible engineering [11].
- ACM Code of Ethics: avoiding harm and respecting privacy [12].
- Professional responsibility: least-privilege access and truthful system reporting.

6.3 Ethical Dilemmas and Justification of Proposed Solution

- **Dilemma:** operational value of location-linked records vs privacy exposure risk.
- **Mitigation:** authenticated APIs, role restrictions, and minimizing sensitive data handling.
- **Dilemma:** false positives causing maintenance overhead.
- **Mitigation:** status workflow and verification steps before repair marking.
- **Dilemma:** AI bias due to dataset domain mismatch (regional road patterns, lighting, weather).
- **Mitigation:** future domain adaptation with Jordan-specific data and periodic model re-evaluation.

6.4 Relevant Environmental Considerations

Better defect lifecycle tracking supports earlier interventions and can reduce repeated road degradation, material waste, and unnecessary rework. Data-driven maintenance planning improves resource usage compared with purely reactive repair cycles.

6.5 Relevance to Jordan and Region (Social, Cultural, Political)

- Supports municipalities with evidence-based maintenance prioritization.
- Improves transparency and public trust in infrastructure servicing processes.
- Aligns with regional digital-government and smart-city modernization goals.

6.6 Other Issues and Constraints

- Operational adoption depends on training and process integration in maintenance units.
- Policy decisions on retention and sharing must be explicit before broad deployment.

Chapter 7: Project Management

7.1 Schedule and Time Management

Phase	Duration	Main Activities
Planning and architecture baseline	Weeks 1-2	Scope definition, initial architecture and stack decision
AI data and model training	Weeks 3-4	Dataset merge, labeling validation, YOLOv8n training, TFLite export
Android AI integration	Weeks 5-6	On-device inference, threshold/NMS tuning, frame processing
Backend foundation	Weeks 7-9	Auth, schema, migrations, core pothole service
Dashboard foundation and operations	Weeks 10-13	Routing, auth flow, list/map/detail pages, bulk/export/stats
Integration, stabilization, and reporting	Weeks 14-16	End-to-end validation, bug fixes, documentation and report finalization

Table 11. Schedule and time management

7.2 Resource and Cost Management

- Primary resources: student laptops, local Docker runtime, open-source frameworks.
- Cost-control strategy: prioritize open-source stack and optional cloud usage.
- Team allocation: backend/database and dashboard components developed in parallel tracks.

7.3 Quality Management

- Type-safe backend/frontend codebases and explicit data contracts.
- Iterative validation through endpoint checks and UI workflow testing.
- Schema consistency through migration snapshots and seeded test accounts.

7.4 Risk Management

Risk	Impact	Mitigation
Duplicate data inflation	Poor analytics quality	Spatial dedup + confirmation counting
Model drift or regional mismatch	Reduced detection quality	Periodic retraining and local dataset expansion
Unauthorized updates	Operational integrity risk	JWT + role-based route restrictions

Risk	Impact	Mitigation
Service startup race conditions	Runtime instability	Healthchecks and compose dependency ordering
User workflow confusion	Low adoption	Simple page navigation and clear status taxonomy

Table 12. Risk register and mitigation plan

7.5 Project Procurement

Procurement needs are minimal for the current implementation-focused phase. The stack relies primarily on open-source software. Optional production procurement includes domain, hosting, and object storage.

Chapter 8: Conclusion and Future Work

8.1 Main Contributions Summary

This work produced a complete AI-enabled implementation-centered pothole operations platform with trained and deployed mobile detection model, secure API, geospatial persistence, and practical dashboard workflows. The report now mirrors actual repository behavior rather than planning-only assumptions.

8.2 Further Future Work

- Add full audit history for status transitions and actor tracking.
- Improve AI recall through local Jordan data collection and domain adaptation fine-tuning.
- Extend AI to multi-class road-damage detection and severity estimation.
- Evaluate quantization and acceleration variants (INT8/NNAPI-specific) for lower-end devices.
- Add geospatial clustering and repair-route recommendation support.
- Expand production deployment hardening (monitoring, alerts, backup routines).
- Integrate municipal GIS and work-order systems through API contracts.
- Add richer operational KPIs and SLA-oriented analytics reporting.

8.3 Lessons Learned

- Early schema stabilization reduces frontend rework significantly.
- Model-size and latency constraints must be considered from the first AI design iteration.
- Operational metrics (precision/recall trade-offs) are as important as raw benchmark scores.
- Role models should be implemented early to avoid insecure shortcuts later.
- Operational UX (bulk actions, filters, exports) matters as much as raw backend completeness.
- Containerized development improves reproducibility and onboarding speed.

References

[1] World Health Organization, "Global Status Report on Road Safety," https://www.who.int/publications/i/item/9789240086517 .

[2] N. Ma et al., "A Review of Vision-Based Pothole Detection Methods Using Computer Vision and Machine Learning," *Sensors*, 2024.

[3] Microsoft, "ASP.NET Core documentation," https://learn.microsoft.com/aspnet/core .

[4] Microsoft, "Entity Framework Core documentation," https://learn.microsoft.com/ef/core .

[5] PostgreSQL Global Development Group, "PostgreSQL Documentation," https://www.postgresql.org/docs/ .

[6] PostGIS Project, "PostGIS Documentation," https://postgis.net/documentation/ .

[7] React Team, "React Documentation," https://react.dev/ .

[8] Vite Team, "Vite Documentation," https://vite.dev/guide/ .

[9] OWASP Foundation, "OWASP Top 10," https://owasp.org/www-project-top-ten/ .

[10] Docker Inc., "Docker Compose Documentation," https://docs.docker.com/compose/ .

[11] IEEE, "IEEE Code of Ethics," https://www.ieee.org/about/corporate/governance/p7-8.html .

[12] ACM, "ACM Code of Ethics and Professional Conduct," https://www.acm.org/code-of-ethics .

[13] NIST, "Cybersecurity Framework and guidance resources," https://www.nist.gov/cyberframework .

[14] International Organization for Standardization (ISO), https://www.iso.org/ .

[15] Jordan Standards and Metrology Organization (JSMO), https://jsmo.gov.jo/ .

[16] Ultralytics, "YOLOv8 Documentation," https://docs.ultralytics.com/ .

[17] TensorFlow, "TensorFlow Lite Guide," https://www.tensorflow.org/lite/guide .

[18] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," 2016.

[19] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," 2020.

[20] Google, "Google Colab Documentation," https://colab.google/ .

APPENDIX A: User Manual

A.0 Quick Start

1. Install Docker Engine + Docker Compose plugin.
2. From repository root, run: `docker compose up --build`.
3. Open dashboard: `http://localhost:4321`.
4. Backend health: `http://localhost:3000/health`.

A.1 Login and Roles

Seed accounts in development environment:

- Admin: `admin@pothole.dev` / `admin123`
- Operator: `operator@pothole.dev` / `operator123`
- Viewer: `viewer@pothole.dev` / `viewer123`

A.2 Daily Workflow for Operations Team

1. Login as operator/admin.
2. Open **Potholes** page and apply status/severity filters.
3. Review unverified/high-confidence records first.
4. Open detail page, inspect location and image, update status/severity.
5. Use bulk actions for batched updates after team review.
6. Export CSV for daily reporting and archival.

A.3 Map Workflow

1. Open **Map View** from sidebar.
2. System loads all available points and fits map bounds automatically.
3. Click marker to see popup summary and navigate to details.
4. Enable heatmap overlay when concentration analysis is needed.

A.4 Vehicle Management

- Admin can add vehicles by name and serial number.
- Admin can activate/deactivate vehicles and remove obsolete entries.
- Vehicle list endpoint returns active vehicles for operations.

A.5 User Management

- Admin can create/edit/delete users and assign roles.
- Operators and viewers cannot access admin-only user routes.

A.6 Troubleshooting

Issue	Possible Cause	Resolution
Cannot login	Invalid credentials or expired token	Re-login with seeded account; clear local storage
No data on dashboard	Backend or DB not healthy	Check compose logs and health endpoint
Map shows fewer points	Filters active or data issue	Clear filters and verify API list response
Upload URL not accessible	Storage base URL misconfiguration	Check backend storage configuration variables

A.7 Guidance on the Use of the Design Consideration Table

Performance: How effectively does the design perform required tasks and remain reliable?

Serviceability: Can maintenance, repair, and updates be applied with reasonable effort?

Economic: Are production and operational costs justified and controlled?

Environmental: What environmental effects exist across usage and lifecycle?

Environmental Sustainability: Does the design support sustainable operation and resources?

Manufacturability: Can the design be produced/deployed and inspected efficiently?

Ethical: Are professional ethics and truthful claims maintained?

Health and safety: Are user and stakeholder safety risks reduced?

Social: Does the solution create positive social impact?

Political: Are governance, policy, and public-sector impacts recognized?

A.8 Incorporating Standards in a Design Project

Representative standards and organizations relevant to this project context include:

1. National Standards System Network (NSSN)
2. ISO (International Organization for Standardization)
3. IEC (International Electrotechnical Commission)
4. NIST guidance resources
5. IEEE Standards Association
6. ASME / ASTM / ANSI references for engineering rigor
7. OWASP security best-practice guidance
8. ACM professional ethics framework
9. Jordan Ministry of Labor occupational safety context
10. Jordan Standards and Metrology Organization (JSMO)